

PRACTICAL WEEK 5

TASK1: Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
GNU nano 7.2 producer_consumer.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include <time.h>

int main() {
    int fd[2];
    pid_t pid;
    char buffer[100];

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid > 0) {
        // Parent - Producer
        close(fd[0]);

        clock_t start = clock();

        for (int i = 1; i <= 5; i++) {
            sprintf(buffer, "Data %d", i);
            write(fd[1], buffer, strlen(buffer) + 1);
            printf("Producer: %s\n", buffer);
        }

        clock_t end = clock();
    }
```

```
GNU nano 7.2 producer_consumer.c *
// Parent - Producer
close(fd[0]);

clock_t start = clock();

for (int i = 1; i <= 5; i++) {
    sprintf(buffer, "Data %d", i);
    write(fd[1], buffer, strlen(buffer) + 1);
    printf("Producer: %s\n", buffer);
}

clock_t end = clock();

close(fd[1]);
wait(NULL);

double time_taken =
    (double)(end - start) / CLOCKS_PER_SEC;

printf("Communication time: %f seconds\n", time_taken);
}
else {
    // Child - Consumer
    close(fd[1]);

    for (int i = 1; i <= 5; i++) {
        read(fd[0], buffer, sizeof(buffer));
        printf("Consumer: %s\n", buffer);
    }

    close(fd[0]);
}

return 0;
}
```

```
jhansi_priya@Jhansi-Priya: ~/ × + v
jhansi_priya@Jhansi-Priya:~$ mkdir practical5
jhansi_priya@Jhansi-Priya:~$ cd practical5
jhansi_priya@Jhansi-Priya:~/practical5$ nano producer_consumer.c
jhansi_priya@Jhansi-Priya:~/practical5$ gcc producer_consumer.c -o producer_consumer
jhansi_priya@Jhansi-Priya:~/practical5$ ./producer_consumer
Producer: Data 1
Producer: Data 2
Producer: Data 3
Producer: Data 4
Producer: Data 5
Consumer: Data 1
Consumer: Data 2
Consumer: Data 2
Consumer: Data 2
Consumer: Data 2
Communication time: 0.000204 seconds
```

TASK2: Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
GNU nano 7.2 pipe_ls_grep.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int fd[2];
    pid_t p1, p2;

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    // First child: ls -l
    p1 = fork();

    if (p1 == -1) {
        perror("fork");
        return 1;
    }

    if (p1 == 0) {
        // Redirect stdout to pipe
        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);

        close(fd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp");
        exit(1);
    }
}
```

```
jhansi_priya@Jhansi-Priya: ~/ × + v
GNU nano 7.2 pipe_ls_grep.c *
    perror("execlp");
    exit(1);
}

// Second child: grep ".c"
p2 = fork();

if (p2 == -1) {
    perror("fork");
    return 1;
}

if (p2 == 0) {
    // Redirect stdin from pipe
    close(fd[1]);

    dup2(fd[0], STDIN_FILENO);

    close(fd[0]);

    execlp("grep", "grep", ".c", NULL);

    perror("execlp");
    exit(1);
}

// Parent
close(fd[0]);
close(fd[1]);

waitpid(p1, NULL, 0);
waitpid(p2, NULL, 0);

return 0;
}
```

```
jhansi_priya@Jhansi-Priya:~/practical5$ ls -l | grep ".c"
-rwxr-xr-x 1 jhansi_priya jhansi_priya 16424 Aug 26 06:47 producer_consumer
-rw-r--r-- 1 jhansi_priya jhansi_priya 1104 Aug 26 06:47 producer_consume.
c
jhansi_priya@Jhansi-Priya:~/practical5$ nano pipe_ls_grep.c
jhansi_priya@Jhansi-Priya:~/practical5$ gcc pipe_ls_grep.c -o pipe_ls_grep
jhansi_priya@Jhansi-Priya:~/practical5$ ./pipe_ls_grep
-rw-r--r-- 1 jhansi_priya jhansi_priya 1015 Aug 26 06:50 pipe_ls_grep.c
-rwxr-xr-x 1 jhansi_priya jhansi_priya 16424 Aug 26 06:47 producer_consumer
-rw-r--r-- 1 jhansi_priya jhansi_priya 1104 Aug 26 06:47 producer_consumer
.c
jhansi_priya@Jhansi-Priya:~/practical5$ |
```